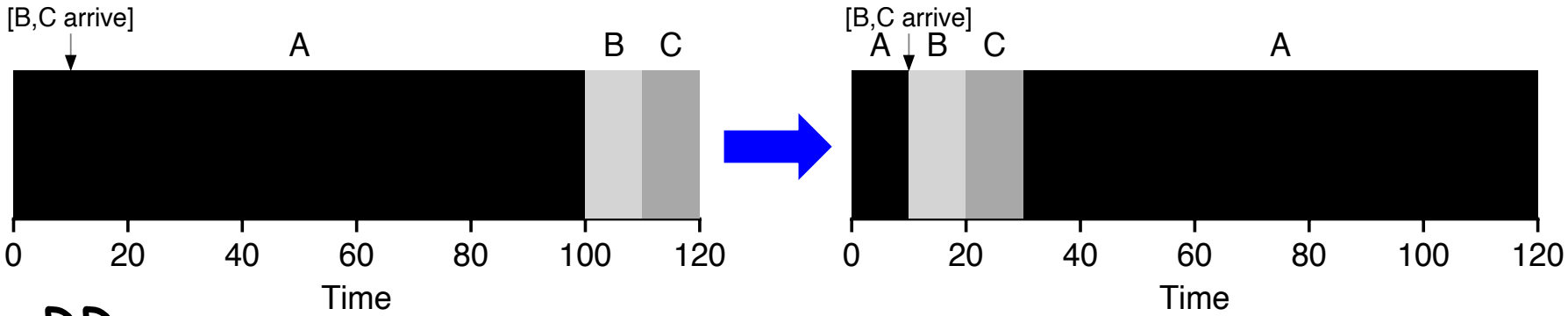
Chapter 8
Multi-Level Feedback Queue

Dong Dai
CIS/UD
dai@udel.edu

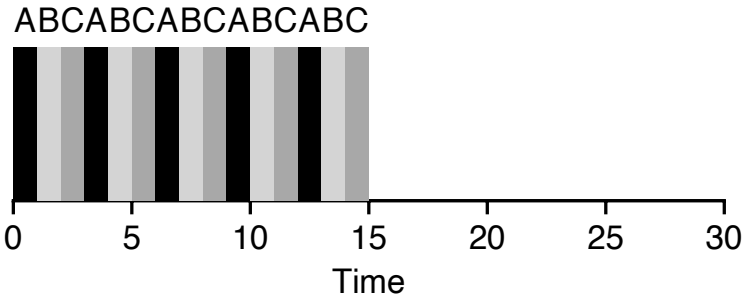
From Chapter 7

Two types of schedulers

- SJF and STCF
  - optimize **turnaround time**, but bad for **response time**

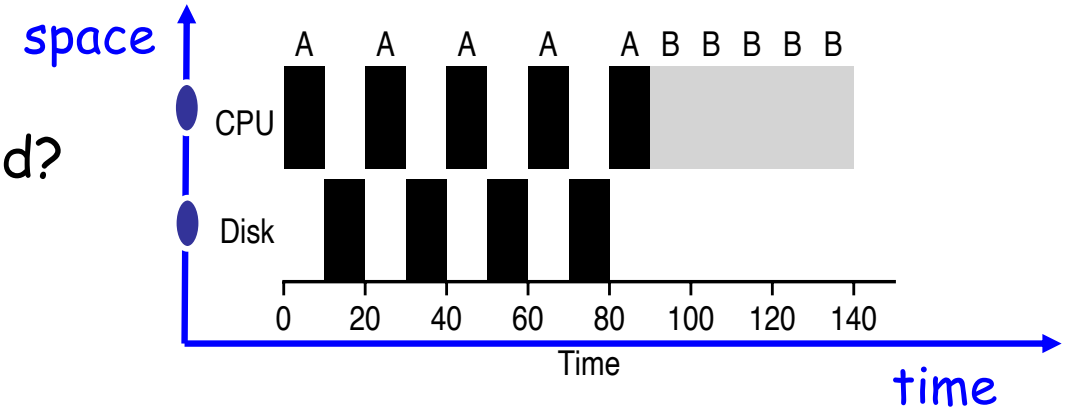


- RR
  - optimize **response time**, but bad for **turnaround time**

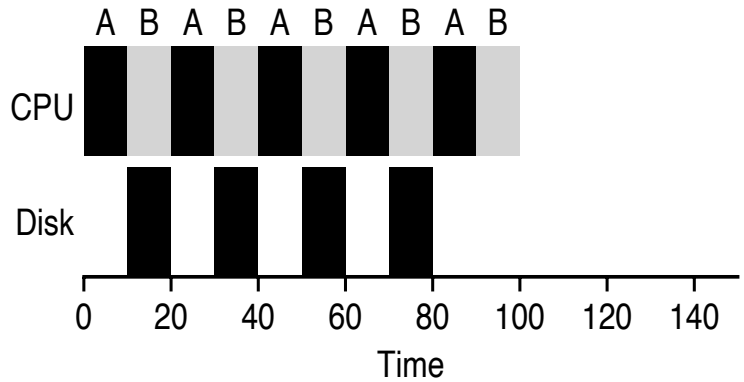


Incorporating I/O

Is this kind of scheduling good?



- **Overlap CPU & I/O**



- Let **interactive jobs** be scheduled (to run) more frequently Is this a bad idea? Not really!
  - while they are performing I/O, other CPU-intensive (**batch**) jobs can run (which is good)

Problems and Solution

- We would like to
  - **optimize turnaround time for batch jobs**
    how? run "shorter" processes first
  - **minimize response time for interactive jobs** how? RR
- But, we don't know anything (e.g., running time, behavior, etc.) about the jobs.....
  - how to achieve these goals **without** perfect knowledge?
  - let scheduler **learn**, as the system runs, the **characteristics** of jobs it is running and thus make better scheduling decisions...
    - **how** to learn and **what** to learn?

The Big Problem

- How to schedule **without** perfect knowledge?
- How can we design a scheduler that both **minimizes** response time for interactive jobs while also **minimizing** turnaround time **without *a priori* knowledge of job length**?
- How/what can scheduler learn?
  - **learn from history……**
    - **learn** from the **past** to **predict** the **future**

Fernando José "Corby" Corbató (July 1, 1926 – July 12, 2019) : MLFQ on CTSS (1962) and **Multics** → Turing Award, 1990

MLFQ

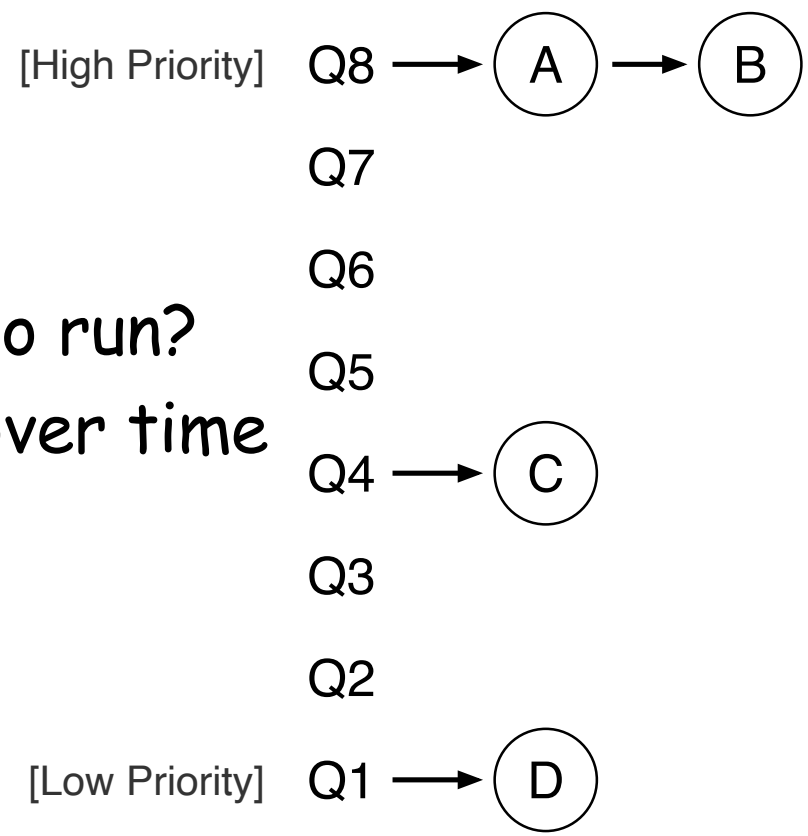
- Multiple queues with different **priority** levels
- MLFQ **varies** the **priority** of a job (**over time**) based on its **observed behavior (history)**
  - job repeatedly relinquishes CPU while waiting for input from keyboard, MLFQ will …….
    - **keep its priority high**
  - job uses the CPU intensively for "long" periods of time, MLFQ will …….
    - **reduce its priority**
  - learn about jobs as they run to predict their **future** behavior
- **Rule 1:** If Priority(A) > Priority(B), A runs (B doesn't)
- **Rule 2:** If Priority(A) = Priority(B), A and B run in RR

MLFQ

Given

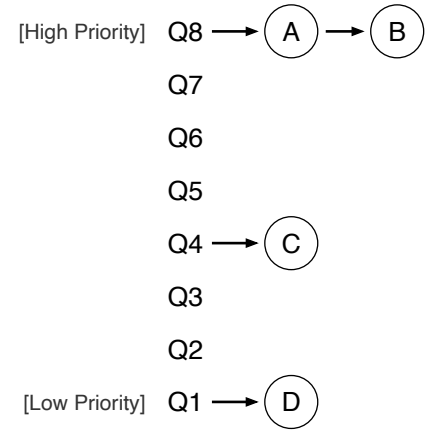
- **Rule 1:** If Priority(A) > Priority(B), A runs (B doesn't)
- **Rule 2:** If Priority(A) = Priority(B), A & B run in RR

would C and D ever run?



- No!
- So how could we make C and D to run?
- Change (**raise**) their priorities over time

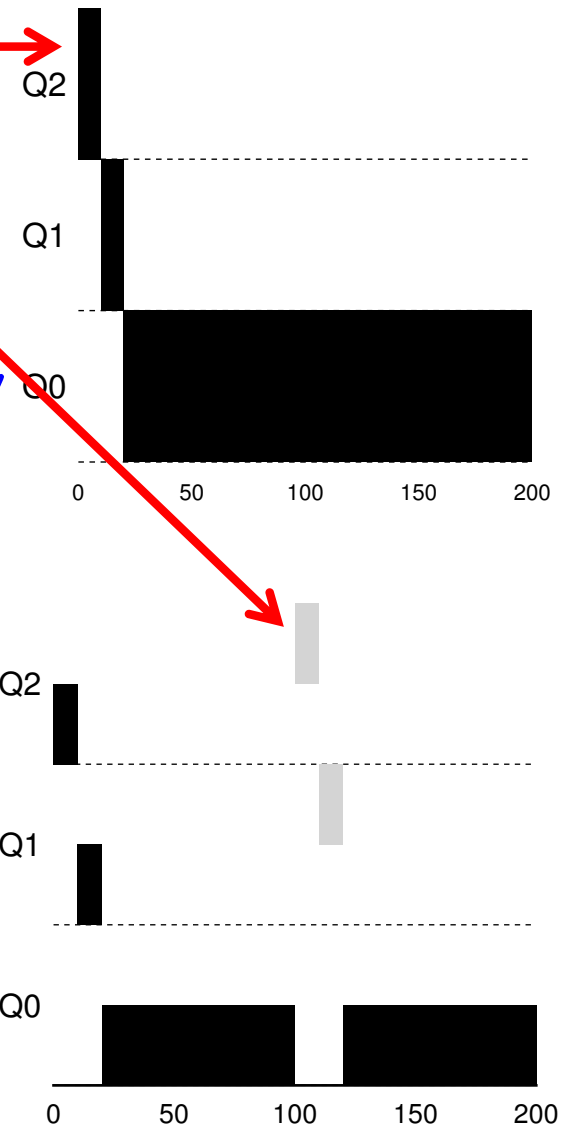
MLFQ



- **Priority changes over time** – how?
- **Workload** - a mix of **interactive jobs** that are short-running (and may frequently relinquish CPU), and some longer-running **CPU-bound (batch) jobs** that need a lot of CPU time but where response time isn't important
- **Rule 3:** When a job first enters system, (where should it go?)
  - it is placed at the **highest** priority (the topmost queue)
- **Rule 4a:** If a job uses up an **entire time slice** while running,
  - its priority is reduced (i.e., it moves down one queue)
- **Rule 4b:** If a job **gives up CPU** before the time slice is up
  - it stays at the same priority level

Examples

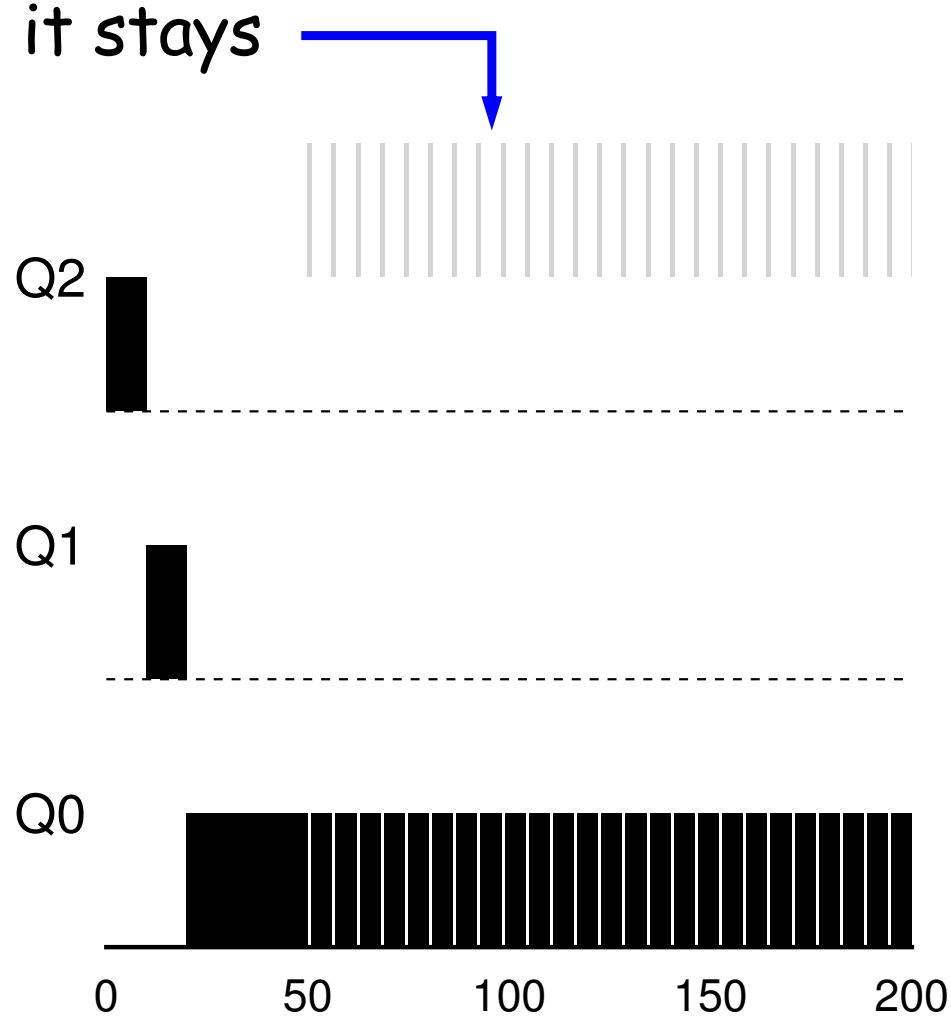
- One long-running job
- Along came a short job at time 100
  - because scheduler doesn't know whether a job is short or long-running, it first **assumes** it is a short job
  - What kind of scheduling is that?
  - approximate SJF?
    - if it actually is a short job, it will run quickly and complete; if it is not a short job, it will slowly move down the queues, and thus soon prove itself to be a long-running job
    - in this manner, **MLFQ approximates SJF**



How about I/O?

- **Rule 4b:** If a job **gives up** CPU before the time slice is up, it stays at the **same** priority level

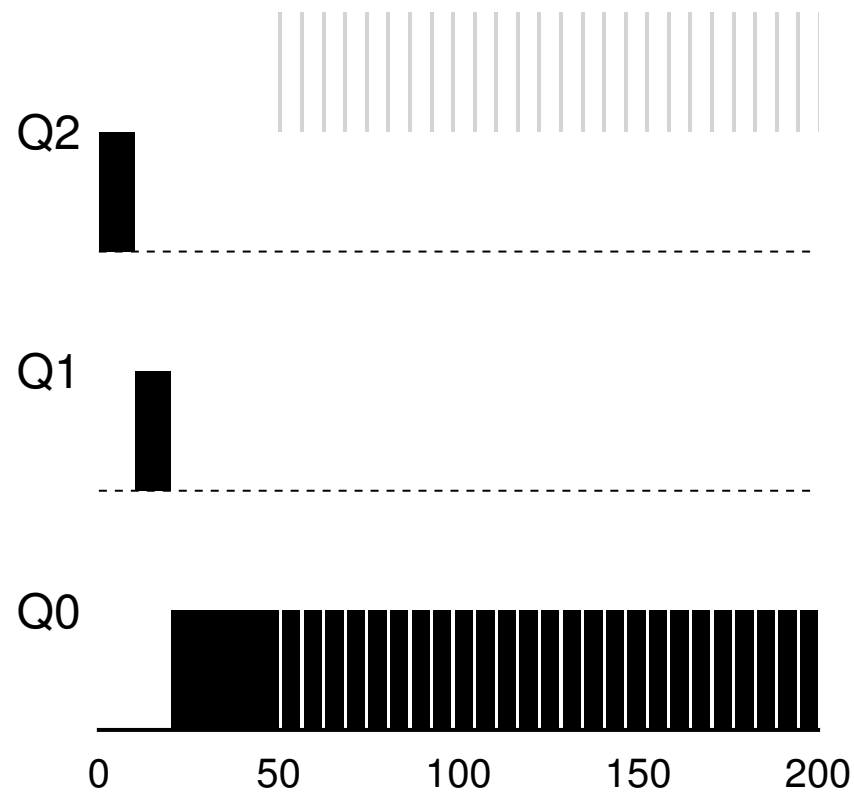
Minimize response time by running interactive jobs quickly



Issues

Any problems???

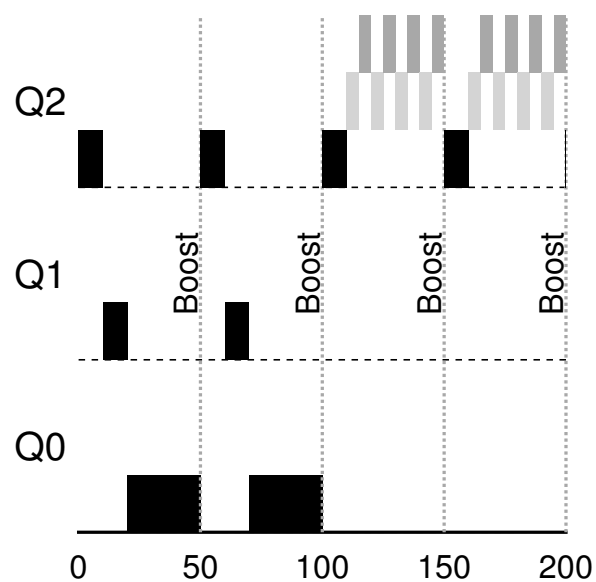
- **starvation** of long running jobs if too many interactive jobs
- **to game (trick) the scheduler:** [doing something sneaky to trick the scheduler into giving you **more than your fair share** of the resource] to remain at the same queue
  - e.g., right before time slice is over, issue an I/O
- **job's changing behavior** (CPU-bound vs. I/O-bound) over time



<u>Scheduling policy</u> becomes a **security** concern (in datacenter)

(1) Deal with Starvation

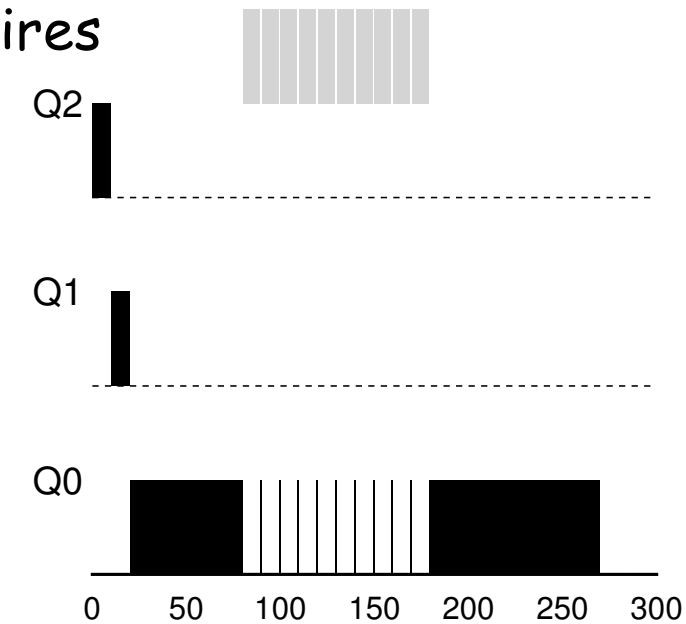
- How to guarantee that CPU-bound jobs make progress ?
- Priority boost
- **Rule 5:** After some time period *S*, move **all** the jobs in the system to the **topmost** queue
  - no starvation
  - accommodate changing behavior



priority boost every 50 ms

(2) Prevent Gaming of Scheduler

- How to prevent "gaming" the scheduler ?
  - dirty trick: a job retains its priority by relinquishing CPU **(right) before** the time slice expires

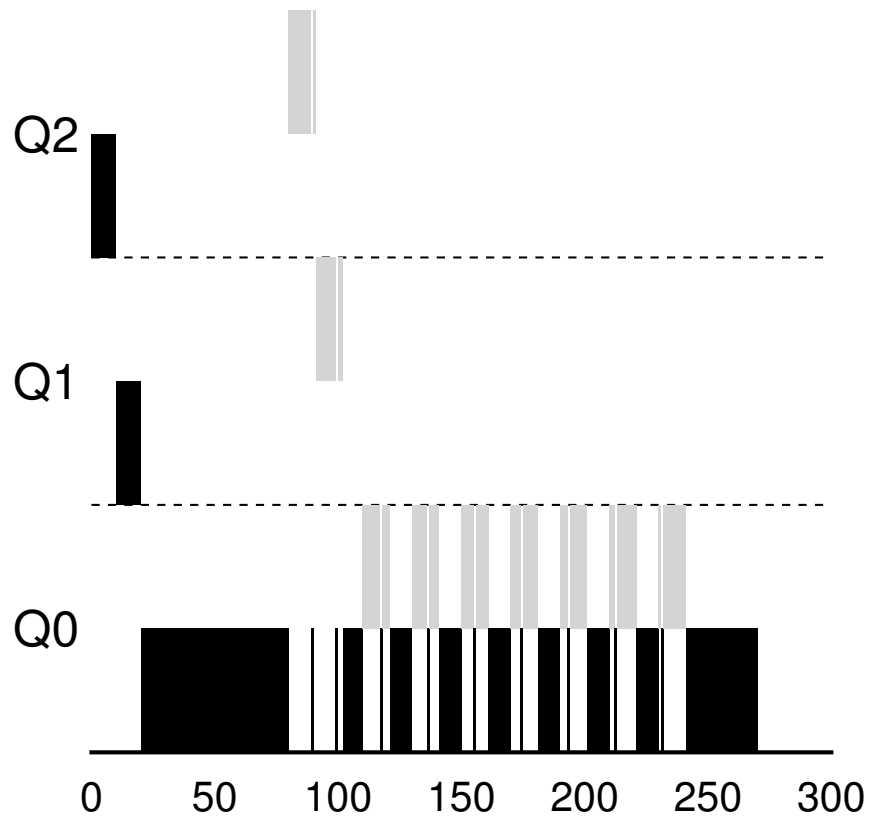


  - perform **better accounting** of CPU time at each level of MLFQ – scheduler keeps track of **how much of a time slice** a job used at a given level; once a job has used its allotment, it is **demoted** to the next priority queue

Better Accounting

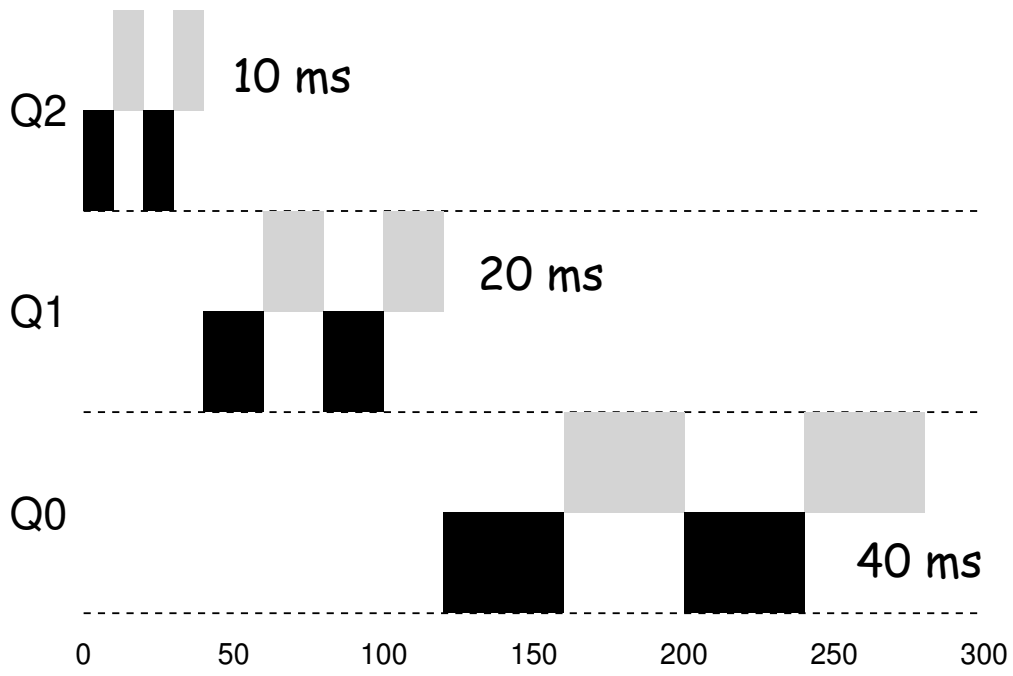
- **Rule 4a:** If a job uses up an entire time slice while running, its priority is **reduced** (i.e., it moves down one queue)
- **Rule 4b:** If a job gives up CPU before the time slice is up, it stays at the **same** priority level

➔**Rule 4 (anti-gaming):** Once a job uses up its **time allotment at a given level** (regardless of how many times it has given up the CPU), its priority is reduced



Tuning MLFQ

- How to parameterize MLFQ ?
  - how many queues?
  - how long should time slice be per queue?
  - how often should priority be boosted?
- No easy answers and need experience with workloads
  - varying time-slice length across different queues
  - e.g., lower priority, longer quanta



Summary – History is the Guide

- **Rule 1:** If Priority(A) > Priority(B), A runs (B doesn't)
- **Rule 2:** If Priority(A) = Priority(B), A and B run in RR
- **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue)
- **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves **down** one queue)
- **Rule 5:** After some time period *S*, move all the jobs in the system to the topmost queue
- Instead of demanding *a priori* knowledge of a job, MLFQ instead observes the execution of a job and prioritizes it accordingly
- MLFQ manages to achieve the best of both worlds: it can deliver excellent overall performance (similar to SJF/STCF) for interactive jobs, and is fair and makes progress for long-running CPU-intensive workloads

Can you answer these?

- True/False
  - With a MLFQ scheduler, compute-bound jobs are given higher priority.
  - With a MLFQ scheduler, high priority jobs have longer time-slices than low priority jobs
- Essay Questions
  - circle the rules that are actually part of the final MLFQ policy
    - If Priority(A) > Priority(B), A runs (B does not)
    - If Priority(A) < Priority(B), A runs (B does not)
    - If Priority(A) = Priority(B), A & B run in round-robin fashion
    - If Priority(A) = Priority(B), A runs to completion, then B
    - When a job enters the system, it is placed at the highest priority (the topmost queue)
    - When a job enters the system, it is placed in any queue
    - When a job enters the system, it is placed in the lowest queue
    - Once a job uses up its time slides at a given level, its priority is reduced
    - Once a job uses up its time slice at a given level, it moves to the end of the round-robin queue
    - Once a job uses up its time slice at a given level, it exits.
    - After some time period, move each job up to a higher priority queue
    - After some time period, move each job down to a lower priority queue
    - After some time period, move all the jobs in the system to the top-most (highest-priority) queue.

Can you answer these?

- True/False
  - With a MLFQ scheduler, compute-bound jobs are given higher priority. **[False]**
  - With a MLFQ scheduler, high priority jobs have longer time-slices than low priority jobs **[False]**
- Essay Questions
  - circle the rules that are actually part of the final MLFQ policy
    - **If Priority(A) > Priority(B), A runs (B does not)**
    - If Priority(A) < Priority(B), A runs (B does not)
    - **If Priority(A) = Priority(B), A & B run in round-robin fashion**
    - If Priority(A) = Priority(B), A runs to completion, then B
    - **When a job enters the system, it is placed at the highest priority (the topmost queue)**
    - When a job enters the system, it is placed in any queue
    - When a job enters the system, it is placed in the lowest queue
    - **Once a job uses up its time slides at a given level, its priority is reduced**
    - Once a job uses up its time slice at a given level, it moves to the end of the round-robin queue
    - Once a job uses up its time slice at a given level, it exits.
    - After some time period, move each job up to a higher priority queue
    - After some time period, move each job down to a lower priority queue
    - **After some time period, move all the jobs in the system to the top-most (highest-priority) queue.**